Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №3 по курсу

«Операционные системы»

Группа: М8О-214БВ-24

Студент: ГеннихА.А.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата:13.10.25

Москва, 2025

Постановка задачи

Вариант 7.

Разработать программу на языке Си, состоящую из двух процессов (родительского и дочернего), которые взаимодействуют через разделяемую память (shared memory) с синхронизацией через семафоры.

Общий метод и алгоритм решения

Использованные системные вызовы:

shm_open() - создает или открывает именованный объект разделяемой памяти (файловый дескриптор для области в RAM)

ftruncate() - устанавливает размер объекта разделяемой памяти (выделяет физическую память под объект)

mmap() - отображает объект разделяемой памяти в адресное пространство процесса (возвращает указатель, по которому можно обращаться к памяти)

munmap() - удаляет отображение разделяемой памяти из адресного пространства процесса

shm_unlink() - удаляет именованный объект разделяемой памяти из системы (освобождает ресурсы)

close() - закрывает файловый дескриптор объекта разделяемой памяти

sem_open() - создает или открывает именованный семафор (счетчик для синхронизации доступа)

sem_wait() - уменьшает счетчик семафора на 1; если счетчик равен 0, процесс блокируется до освобождения ресурса

sem_post() - увеличивает счетчик семафора на 1; если есть процессы, ожидающие на sem_wait, один из них просыпается

sem_close() - закрывает семафор в текущем процессе (освобождает дескриптор)

sem_unlink() - удаляет именованный семафор из системы

void cleanup() - Последовательно освобождает: Отключение shared memory (munmap) Закрытие файлового дескриптора (close) Удаление объекта shared memory (shm_unlink) Удаление семафоров (sem_unlink)

error_exit() - Обрабатывает ошибки и завершает программу

В работе используется подход межпроцессного взаимодействия через именованную разделяемую память (POSIX shared memory) с синхронизацией доступа посредством именованных семафоров. Родительский процесс создает все необходимые объекты IPC с уникальными именами на основе своего PID, что позволяет изолировать разные экземпляры программы друг от друга. После создания разделяемой памяти (shm_open, ftruncate, mmap) и двух семафоров (один для синхронизации записи, другой для чтения), родитель порождает дочерний процесс через fork. В дочернем процессе стандартный ввод перенаправляется на файл с числами (freopen), после чего образ процесса заменяется программой child (execl), которой передаются имена созданных объектов. Дочерний процесс подключается к существующей разделяемой памяти и семафорам, читает числа из перенаправленного stdin, вычисляет их сумму и через механизм семафоров (sem_wait/sem_post) синхронизированно записывает результат в разделяемую память. Родитель, ожидая на семафоре, получает сигнал о готовности данных, читает результат из общей памяти, выводит его на экран и выполняет полную очистку всех созданных объектов (munmap, shm_unlink, sem_unlink). После парсинга аргументов командной строки программа генерирует случайный набор точек и

инициализирует первые K точек как начальные центры. Далее запускается итеративный процесс: на каждом шаге массив точек делится на равные части по числу потоков, создаётся заданное количество потоков через системный вызов pthread_create, каждый поток обрабатывает свой диапазон точек, определяя для каждой ближайший центр и при необходимости поднимая атомарный флаг changed. После завершения всех потоков (ожидание через pthread_join) главный поток пересчитывает центры функцией recalculate_centers и проверяет флаг — если изменений не было, алгоритм завершается.

Код программы

Parent.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <semaphore.h>
#include <signal.h>
#include <errno.h>

#define SHM_SIZE 4096

typedef struct {
    float result;
    int data_ready;
    int child_done;
} shared_data_t;

void error_exit(const char *text) {
    perror(text);
    exit(EXIT_FAILURE);
}

void cleanup(const char *shm_name, const char *sem_parent_name,
             const char *sem_child_name, void *shm_ptr, int shm_fd) {
    if (shm_ptr != NULL && shm_ptr != MAP_FAILED) {
        munmap(shm_ptr, SHM_SIZE);
    }
    if (shm_fd >= 0) {
        close(shm_fd);
    }
    if (shm_name != NULL) {
        shm_unlink(shm_name);
    }
    if (sem_parent_name != NULL) {
        sem_unlink(sem_parent_name);
    }
    if (sem_child_name != NULL) {
```

```
        sem_unlink(sem_child_name);
    }
}

int main() {
    pid_t pid;
    char filename[256];
    char shm_name[256];
    char sem_parent_name[256];
    char sem_child_name[256];

    sprintf(shm_name, "/shm_%d", getpid());
    sprintf(sem_parent_name, "/sem_parent_%d", getpid());
    sprintf(sem_child_name, "/sem_child_%d", getpid());

    printf("Родительский процесс PID: %d\n", getpid());
    printf("Имя shared memory: %s\n", shm_name);
    printf("Имя семафора родителя: %s\n", sem_parent_name);
    printf("Имя семафора ребенка: %s\n", sem_child_name);

    printf("Введите имя файла: ");
    if (fgets(filename, sizeof(filename), stdin) == NULL) {
        error_exit("fgets");
    }

    filename[strcspn(filename, "\n")] = '\0';

    if (strlen(filename) == 0) {
        fprintf(stderr, "Введите имя файла\n");
        exit(EXIT_FAILURE);
    }

    int shm_fd = shm_open(shm_name, O_CREAT | O_RDWR, 0666);
    if (shm_fd == -1) {
        error_exit("shm_open");
    }

    if (ftruncate(shm_fd, SHM_SIZE) == -1) {
        cleanup(shm_name, NULL, NULL, NULL, shm_fd);
        error_exit("Уменьшите файл");
    }

    void *shm_ptr = mmap(NULL, SHM_SIZE, PROT_READ | PROT_WRITE,
```

```
                    MAP_SHARED, shm_fd, 0);
    if (shm_ptr == MAP_FAILED) {
        cleanup(shm_name, NULL, NULL, NULL, shm_fd);
        error_exit(MAP_FAILED);
    }

    shared_data_t *shared_data = (shared_data_t *)shm_ptr;
    shared_data->data_ready = 0;
    shared_data->child_done = 0;

    sem_t *sem_parent = sem_open(sem_parent_name, O_CREAT, 0666, 0);
    if (sem_parent == SEM_FAILED) {
        cleanup(shm_name, NULL, NULL, shm_ptr, shm_fd);
        error_exit("sem_open parent");
    }

    sem_t *sem_child = sem_open(sem_child_name, O_CREAT, 0666, 1);
    if (sem_child == SEM_FAILED) {
        sem_unlink(sem_parent_name);
        cleanup(shm_name, sem_parent_name, NULL, shm_ptr, shm_fd);
        error_exit("sem_open child");
    }

    sem_close(sem_parent);
    sem_close(sem_child);

    pid = fork();
    if (pid == -1) {
        cleanup(shm_name, sem_parent_name, sem_child_name, shm_ptr, shm_fd);
        error_exit("fork");
    }

    if (pid == 0) {
        FILE *file = freopen(filename, "r", stdin);
        if (file == NULL) {
            fprintf(stderr, "Не удалось открыть файл ребенка %s\n", filename);
            exit(EXIT_FAILURE);
        }

        execl("./child", "child", shm_name, sem_parent_name, sem_child_name,
NULL);

        perror("execl");
```

```
        exit(EXIT_FAILURE);
    } else {
        printf("Дочерний процесс создан (PID: %d)\n", pid);

        sem_parent = sem_open(sem_parent_name, 0);
        if (sem_parent == SEM_FAILED) {
            cleanup(shm_name, sem_parent_name, sem_child_name, shm_ptr,
shm_fd);
            error_exit("sem_open parent in parent process");
        }

        if (sem_wait(sem_parent) == -1) {
            sem_close(sem_parent);
            cleanup(shm_name, sem_parent_name, sem_child_name, shm_ptr,
shm_fd);
            error_exit("sem_wait parent");
        }

        if (shared_data->data_ready) {
            printf("Получен результат: %.2f\n", shared_data->result);
        }

        sem_post(sem_child);

        wait(NULL);

        sem_close(sem_parent);

        cleanup(shm_name, sem_parent_name, sem_child_name, shm_ptr, shm_fd);
    }

    return 0;
}
```

Child.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <semaphore.h>
```

```
#include <errno.h>

#define SHM_SIZE 4096

typedef struct {
    float result;
    int data_ready;
    int child_done;
} shared_data_t;

void error_exit(const char *msg) {
    perror(msg);
    exit(EXIT_FAILURE);
}

int main(int argc, char *argv[]) {
    if (argc != 4) {
        fprintf(stderr, "Использование: %s <shm_name> <sem_parent_name>
<sem_child_name>\n", argv[0]);
        exit(EXIT_FAILURE);
    }

    char *shm_name = argv[1];
    char *sem_parent_name = argv[2];
    char *sem_child_name = argv[3];

    printf("Дочерний процесс PID: %d\n", getpid());
    printf("Shared memory: %s\n", shm_name);
    printf("Семафор родителя: %s\n", sem_parent_name);
    printf("Семафор ребенка: %s\n", sem_child_name);

    int shm_fd = shm_open(shm_name, O_RDWR, 0666);
    if (shm_fd == -1) {
        error_exit("child: shm_open");
    }

    void *shm_ptr = mmap(NULL, SHM_SIZE, PROT_READ | PROT_WRITE,
                         MAP_SHARED, shm_fd, 0);
    if (shm_ptr == MAP_FAILED) {
        close(shm_fd);
        error_exit("child: mmap");
    }
```

```
shared_data_t *shared_data = (shared_data_t *)shm_ptr;

sem_t *sem_parent = sem_open(sem_parent_name, 0);
if (sem_parent == SEM_FAILED) {
    munmap(shm_ptr, SHM_SIZE);
    close(shm_fd);
    error_exit("child: sem_open parent");
}

sem_t *sem_child = sem_open(sem_child_name, 0);
if (sem_child == SEM_FAILED) {
    sem_close(sem_parent);
    munmap(shm_ptr, SHM_SIZE);
    close(shm_fd);
    error_exit("child: sem_open child");
}


float sum = 0.0;
float num;
int count = 0;
char line[256];

while (fgets(line, sizeof(line), stdin) != NULL) {
    char *token = strtok(line, " \t\n");
    while (token != NULL) {
        char *endptr;
        num = strtof(token, &endptr);
        if (*endptr == '\0') {
            sum += num;
            count++;
            printf("Прочитано число %.2f, сумма = %.2f\n", num, sum);
        }
        token = strtok(NULL, " \t\n");
    }
}

printf("Всего чисел: %d, сумма = %.2f\n", count, sum);

if (sem_wait(sem_child) == -1) {
    perror("child: sem_wait child");
} else {
    shared_data->result = sum;
```

```
        shared_data->data_ready = 1;

        printf("Результат записан в shared memory\n");

        if (sem_post(sem_parent) == -1) {
            perror("child: sem_post parent");
        }
    }

    sem_close(sem_parent);
    sem_close(sem_child);
    munmap(shm_ptr, SHM_SIZE);
    close(shm_fd);

    printf("Дочерниый процесс завершил работу\n");

    return 0;
}
```

Протокол работы программы

Тестирование:

Data.txt:

10.5 20.3 30.7
5.2 15.8
25.1

```
hoodrich@DESKTOP-DUQ2OCS:~/OSI_LAB/laba_3$ ./parent
Родительский процесс PID: 8904
Имя shared memory: /shm_8904
Имя семафора родителя: /sem_parent_8904
Имя семафора ребенка: /sem_child_8904
Введите имя файла: data.txt
Дочерний процесс создан (PID: 8911)
Дочерний процесс PID: 8911
Shared memory: /shm_8904
Семафор родителя: /sem_parent_8904
Семафор ребенка: /sem_child_8904
Прочитано число 10.50, сумма = 10.50
Прочитано число 20.30, сумма = 30.80
Прочитано число 30.70, сумма = 61.50
Прочитано число 5.20, сумма = 66.70
Прочитано число 15.80, сумма = 82.50
Прочитано число 25.10, сумма = 107.60
Всего чисел: 6, сумма = 107.60
```

Результат записан в shared memory
Дочерний процесс завершил работу
Получен результат: 107.60

Strace:

```
hoodrich@DESKTOP-DUQ2OCS:~/OSI_LAB/laba_3$ strace -f ./parent
execve("./parent", ["./parent"], 0x7ffc6fe17898 /* 36 vars */) = 0
brk(NULL)                               = 0x5647a388a000
mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) =
0x7fab4bb59000
access("/etc/ld.so.preload", R_OK)      = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
fstat(3, {st_mode=S_IFREG|0644, st_size=20203, ...}) = 0
mmap(NULL, 20203, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7fab4bb54000
close(3)                                = 0
openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\220\243\2\0\0\0\0\0"..., 832) = 832
pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"..., 784, 64) = 784
fstat(3, {st_mode=S_IFREG|0755, st_size=2125328, ...}) = 0
pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"..., 784, 64) = 784
mmap(NULL, 2170256, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x7fab4b942000
mmap(0x7fab4b96a000, 1605632, PROT_READ|PROT_EXEC,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x28000) = 0x7fab4b96a000
mmap(0x7fab4baf2000, 323584, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3,
0x1b0000) = 0x7fab4baf2000
mmap(0x7fab4bb41000, 24576, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1fe000) = 0x7fab4bb41000
mmap(0x7fab4bb47000, 52624, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0x7fab4bb47000
close(3)                                = 0
mmap(NULL, 12288, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) =
0x7fab4b93f000
arch_prctl(ARCH_SET_FS, 0x7fab4b93f740) = 0
set_tid_address(0x7fab4b93fa10)         = 9210
set_robust_list(0x7fab4b93fa20, 24)     = 0
rseq(0x7fab4b940060, 0x20, 0, 0x53053053) = 0
mprotect(0x7fab4bb41000, 16384, PROT_READ) = 0
mprotect(0x56478d245000, 4096, PROT_READ) = 0
mprotect(0x7fab4bb91000, 8192, PROT_READ) = 0
prlimit64(0, RLIMIT_STACK, NULL, {rlim_cur=8192*1024, rlim_max=RLIM64_INFINITY}) = 0
munmap(0x7fab4bb54000, 20203)           = 0
getpid()                                = 9210
getpid()                                = 9210
getpid()                                = 9210
getpid()                                = 9210
fstat(1, {st_mode=S_IFCHR|0620, st_rdev=makedev(0x88, 0x4), ...}) = 0
getrandom("\x35\xbd\x5e\x8e\xd2\x00\x12\xda", 8, GRND_NONBLOCK) = 8
brk(NULL)                               = 0x5647a388a000
brk(0x5647a38ab000)                     = 0x5647a38ab000
write(1,
"\320\240\320\276\320\264\320\270\321\202\320\265\320\273\321\214\321\201\320\272\320\270\320\271
\320\277\321\200\320\276\321"..., 50Родительский процесс PID: 9210
) = 50
write(1, "\320\230\320\274\321\217 shared memory: /shm_9210\n", 32Имя shared memory: /shm_9210
) = 32
```

write(1, "\320\230\320\274\321\217 \321\201\320\265\320\274\320\260\321\204\320\276\321\200\320\260 \321\200\320\276\320\264\320\270"..., 59Имя семафора родителя: /sem_parent_9210
) = 59
write(1, "\320\230\320\274\321\217 \321\201\320\265\320\274\320\260\321\204\320\276\321\200\320\260 \321\200\320\265\320\261\320\265"..., 56Имя семафора ребенка: /sem_child_9210
) = 56
fstat(0, {st_mode=S_IFCHR|0620, st_rdev=makedev(0x88, 0x4), ...}) = 0
write(1, "\320\222\320\262\320\265\320\264\320\270\321\202\320\265 \320\270\320\274\321\217 \321\204\320\260\320\271\320\273\320\260"..., 34Введите имя файла:) = 34
read(0, data.txt
"data.txt\n", 1024) = 9
openat(AT_FDCWD, "/dev/shm/shm_9210", O_RDWR|O_CREAT|O_NOFOLLOW|O_CLOEXEC, 0666) = 3
ftruncate(3, 4096) = 0
mmap(NULL, 4096, PROT_READ|PROT_WRITE, MAP_SHARED, 3, 0) = 0x7fab4bb58000
openat(AT_FDCWD, "/dev/shm/sem.sem_parent_9210", O_RDWR|O_NOFOLLOW|O_CLOEXEC) = -1 ENOENT (No such file or directory)
getrandom("\x42\x22\x4b\x43\xaf\xe1\xfb\x2c", 8, GRND_NONBLOCK) = 8
newfstatat(AT_FDCWD, "/dev/shm/sem.yrXnz3", 0x7ffdd38504e0, AT_SYMLINK_NOFOLLOW) = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/dev/shm/sem.yrXnz3", O_RDWR|O_CREAT|O_EXCL|O_NOFOLLOW|O_CLOEXEC, 0666) = 4
write(4, "\0\0\0\0\0\0\0\0\200\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0", 32) = 32
mmap(NULL, 32, PROT_READ|PROT_WRITE, MAP_SHARED, 4, 0) = 0x7fab4bb57000
link("/dev/shm/sem.yrXnz3", "/dev/shm/sem.sem_parent_9210") = 0
fstat(4, {st_mode=S_IFREG|0644, st_size=32, ...}) = 0
unlink("/dev/shm/sem.yrXnz3") = 0
close(4) = 0
openat(AT_FDCWD, "/dev/shm/sem.sem_child_9210", O_RDWR|O_NOFOLLOW|O_CLOEXEC) = -1 ENOENT (No such file or directory)
getrandom("\x1a\x14\xc0\x9b\x6c\xfd\xf7\xa5", 8, GRND_NONBLOCK) = 8
newfstatat(AT_FDCWD, "/dev/shm/sem.6YLAwt", 0x7ffdd38504e0, AT_SYMLINK_NOFOLLOW) = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/dev/shm/sem.6YLAwt", O_RDWR|O_CREAT|O_EXCL|O_NOFOLLOW|O_CLOEXEC, 0666) = 4
write(4, "\1\0\0\0\0\0\0\0\200\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0", 32) = 32
mmap(NULL, 32, PROT_READ|PROT_WRITE, MAP_SHARED, 4, 0) = 0x7fab4bb56000
link("/dev/shm/sem.6YLAwt", "/dev/shm/sem.sem_child_9210") = 0
fstat(4, {st_mode=S_IFREG|0644, st_size=32, ...}) = 0
unlink("/dev/shm/sem.6YLAwt") = 0
close(4) = 0
clone(child_stack=NULL, flags=CLONE_CHILD_CLEARTID|CLONE_CHILD_SETTID|SIGCHLDstrace: Process 9238 attached
, child_tidptr=0x7fab4b93fa10) = 9238
[pid 9238] **set_robust_list**(0x7fab4b93fa20, 24 <unfinished ...>
[pid 9210] **write**(1, "\320\224\320\276\321\207\320\265\321\200\320\275\320\270\320\271 \320\277\321\200\320\276\321\206\320\265\321\201\321\201 "..., 57 <unfinished ...>
Дочерний процесс создан (PID: 9238)
[pid 9238] <... set_robust_list resumed>) = 0
[pid 9210] <... write resumed>) = 57
[pid 9210] **futex**(0x7fab4bb57000, FUTEX_WAIT_BITSET|FUTEX_CLOCK_REALTIME, 0, NULL, FUTEX_BITSET_MATCH_ANY <unfinished ...>
[pid 9238] **openat**(AT_FDCWD, "data.txt", O_RDONLY) = 4
[pid 9238] **dup3**(4, 0, 0) = 0
[pid 9238] **close**(4) = 0
[pid 9238] **execve**("./child", ["child", "/shm_9210", "/sem_parent_9210", "/sem_child_9210"], 0x7ffdd3850dd8 /* 36 vars */) = 0

```
[pid  9238] brk(NULL)                   = 0x55e17086f000
[pid  9238] mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1,
0) = 0x7fc1969fc000
[pid  9238] access("/etc/ld.so.preload", R_OK) = -1 ENOENT (No such file or directory)
[pid  9238] openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
[pid  9238] fstat(3, {st_mode=S_IFREG|0644, st_size=20203, ...}) = 0
[pid  9238] mmap(NULL, 20203, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7fc1969f7000
[pid  9238] close(3)                    = 0
[pid  9238] openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
[pid  9238] read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\220\243\2\0\0\0\0\0"..., 832) = 832
[pid  9238] pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"..., 784, 64) = 784
[pid  9238] fstat(3, {st_mode=S_IFREG|0755, st_size=2125328, ...}) = 0
[pid  9238] pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"..., 784, 64) = 784
[pid  9238] mmap(NULL, 2170256, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) =
0x7fc1967e5000
[pid  9238] mmap(0x7fc19680d000, 1605632, PROT_READ|PROT_EXEC,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x28000) = 0x7fc19680d000
[pid  9238] mmap(0x7fc196995000, 323584, PROT_READ,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1b0000) = 0x7fc196995000
[pid  9238] mmap(0x7fc1969e4000, 24576, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1fe000) = 0x7fc1969e4000
[pid  9238] mmap(0x7fc1969ea000, 52624, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0x7fc1969ea000
[pid  9238] close(3)                    = 0
[pid  9238] mmap(NULL, 12288, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -
1, 0) = 0x7fc1967e2000
[pid  9238] arch_prctl(ARCH_SET_FS, 0x7fc1967e2740) = 0
[pid  9238] set_tid_address(0x7fc1967e2a10) = 9238
[pid  9238] set_robust_list(0x7fc1967e2a20, 24) = 0
[pid  9238] rseq(0x7fc1967e3060, 0x20, 0, 0x53053053) = 0
[pid  9238] mprotect(0x7fc1969e4000, 16384, PROT_READ) = 0
[pid  9238] mprotect(0x55e1604bb000, 4096, PROT_READ) = 0
[pid  9238] mprotect(0x7fc196a34000, 8192, PROT_READ) = 0
[pid  9238] prlimit64(0, RLIMIT_STACK, NULL, {rlim_cur=8192*1024, rlim_max=RLIM64_INFINITY})
= 0
[pid  9238] munmap(0x7fc1969f7000, 20203) = 0
[pid  9238] getpid()                    = 9238
[pid  9238] fstat(1, {st_mode=S_IFCHR|0620, st_rdev=makedev(0x88, 0x4), ...}) = 0
[pid  9238] getrandom("\x11\x98\xcd\x0c\x78\x10\xbe\xbb", 8, GRND_NONBLOCK) = 8
[pid  9238] brk(NULL)                   = 0x55e17086f000
[pid  9238] brk(0x55e170890000)         = 0x55e170890000
[pid  9238] write(1, "\320\224\320\276\321\207\320\265\321\200\320\275\320\270\320\271
\320\277\321\200\320\276\321\206\320\265\321\201\321\201 "..., 42Дочерний процесс PID: 9238
) = 42
[pid  9238] write(1, "Shared memory: /shm_9210\n", 25Shared memory: /shm_9210
) = 25
[pid  9238] write(1, "\320\241\320\265\320\274\320\260\321\204\320\276\321\200
\321\200\320\276\320\264\320\270\321\202\320\265\320\273\321\217:"..., 50Семафор родителя:
/sem_parent_9210
) = 50
[pid  9238] write(1, "\320\241\320\265\320\274\320\260\321\204\320\276\321\200
\321\200\320\265\320\261\320\265\320\275\320\272\320\260: /"..., 47Семафор ребенка: /sem_child_9210
) = 47
[pid  9238] openat(AT_FDCWD, "/dev/shm/shm_9210", O_RDWR|O_NOFOLLOW|O_CLOEXEC) = 3
[pid  9238] mmap(NULL, 4096, PROT_READ|PROT_WRITE, MAP_SHARED, 3, 0) = 0x7fc1969fb000
[pid  9238] openat(AT_FDCWD, "/dev/shm/sem.sem_parent_9210",
O_RDWR|O_NOFOLLOW|O_CLOEXEC) = 4
```

```
[pid  9238] fstat(4, {st_mode=S_IFREG|0644, st_size=32, ...}) = 0
[pid  9238] mmap(NULL, 32, PROT_READ|PROT_WRITE, MAP_SHARED, 4, 0) = 0x7fc1969fa000
[pid  9238] close(4)                    = 0
[pid  9238] openat(AT_FDCWD, "/dev/shm/sem.sem_child_9210",
O_RDWR|O_NOFOLLOW|O_CLOEXEC) = 4
[pid  9238] fstat(4, {st_mode=S_IFREG|0644, st_size=32, ...}) = 0
[pid  9238] mmap(NULL, 32, PROT_READ|PROT_WRITE, MAP_SHARED, 4, 0) = 0x7fc1969f9000
[pid  9238] close(4)                    = 0
[pid  9238] fstat(0, {st_mode=S_IFREG|0644, st_size=29, ...}) = 0
[pid  9238] read(0, "10.5 20.3 30.7\n5.2 15.8\n25.1\n", 4096) = 29
[pid  9238] write(1, "\320\237\321\200\320\276\321\207\320\270\321\202\320\260\320\275\320\276
\321\207\320\270\321\201\320\273\320\276 10"..., 56Прочитано число 10.50, сумма = 10.50
) = 56
[pid  9238] write(1, "\320\237\321\200\320\276\321\207\320\270\321\202\320\260\320\275\320\276
\321\207\320\270\321\201\320\273\320\276 20"..., 56Прочитано число 20.30, сумма = 30.80
) = 56
[pid  9238] write(1, "\320\237\321\200\320\276\321\207\320\270\321\202\320\260\320\275\320\276
\321\207\320\270\321\201\320\273\320\276 30"..., 56Прочитано число 30.70, сумма = 61.50
) = 56
[pid  9238] write(1, "\320\237\321\200\320\276\321\207\320\270\321\202\320\260\320\275\320\276
\321\207\320\270\321\201\320\273\320\276 5."..., 55Прочитано число 5.20, сумма = 66.70
) = 55
[pid  9238] write(1, "\320\237\321\200\320\276\321\207\320\270\321\202\320\260\320\275\320\276
\321\207\320\270\321\201\320\273\320\276 15"..., 56Прочитано число 15.80, сумма = 82.50
) = 56
[pid  9238] write(1, "\320\237\321\200\320\276\321\207\320\270\321\202\320\260\320\275\320\276
\321\207\320\270\321\201\320\273\320\276 25"..., 57Прочитано число 25.10, сумма = 107.60
) = 57
[pid  9238] read(0, "", 4096)           = 0
[pid  9238] write(1, "\320\222\321\201\320\265\320\263\320\276
\321\207\320\270\321\201\320\265\320\273: 6, \321\201\321\203\320\274"..., 46Всего чисел: 6, сумма =
107.60
) = 46
[pid  9238] write(1, "\320\240\320\265\320\267\321\203\320\273\321\214\321\202\320\260\321\202
\320\267\320\260\320\277\320\270\321\201\320\260\320"..., 51Результат записан в shared memory
) = 51
[pid  9238] futex(0x7fc1969fa000, FUTEX_WAKE, 1 <unfinished ...>
[pid  9210] <... futex resumed>)        = 0
[pid  9238] <... futex resumed>)        = 1
[pid  9210] write(1, "\320\237\320\276\320\273\321\203\321\207\320\265\320\275
\321\200\320\265\320\267\321\203\320\273\321\214\321\202\320\260\321"..., 42 <unfinished ...>
Получен результат: 107.60
[pid  9238] munmap(0x7fc1969fa000, 32 <unfinished ...>
[pid  9210] <... write resumed>)        = 42
[pid  9238] <... munmap resumed>)       = 0
[pid  9210] wait4(-1,  <unfinished ...>
[pid  9238] munmap(0x7fc1969f9000, 32)  = 0
[pid  9238] munmap(0x7fc1969fb000, 4096) = 0
[pid  9238] close(3)                    = 0
[pid  9238] write(1, "\320\224\320\276\321\207\320\265\321\200\320\275\320\270\320\271
\320\277\321\200\320\276\321\206\320\265\321\201\321\201 "..., 62Дочерний процесс завершил работу
) = 62
[pid  9238] exit_group(0)               = ?
[pid  9238] +++ exited with 0 +++
<... wait4 resumed>NULL, 0, NULL)       = 9238
--- SIGCHLD {si_signo=SIGCHLD, si_code=CLD_EXITED, si_pid=9238, si_uid=1000, si_status=0,
si_utime=0, si_stime=1 /* 0.01 s */} ---
```

```
munmap(0x7fab4bb57000, 32)          = 0
munmap(0x7fab4bb56000, 32)          = 0
munmap(0x7fab4bb58000, 4096)        = 0
close(3)                            = 0
unlink("/dev/shm/shm_9210")         = 0
unlink("/dev/shm/sem.sem_parent_9210")  = 0
unlink("/dev/shm/sem.sem_child_9210")   = 0
exit_group(0)                       = ?
+++ exited with 0 +++
```

Вывод

В ходе выполнения работы были приобретены навыки управления процессами в Linux и организации межпроцессного взаимодействия через разделяемую память (shm_open, mmap) с синхронизацией семафорами (sem_open, sem_wait, sem_post). Разработана программа из двух процессов: родитель создает уникальные объекты, запускает дочерний процесс через fork/execl, ребенок читает числа из файла, вычисляет сумму и через семафоры передает результат в shared memory родителю. Все системные вызовы обрабатываются с проверкой ошибок, ресурсы гарантированно освобождаются (munmap, shm_unlink, sem_unlink).